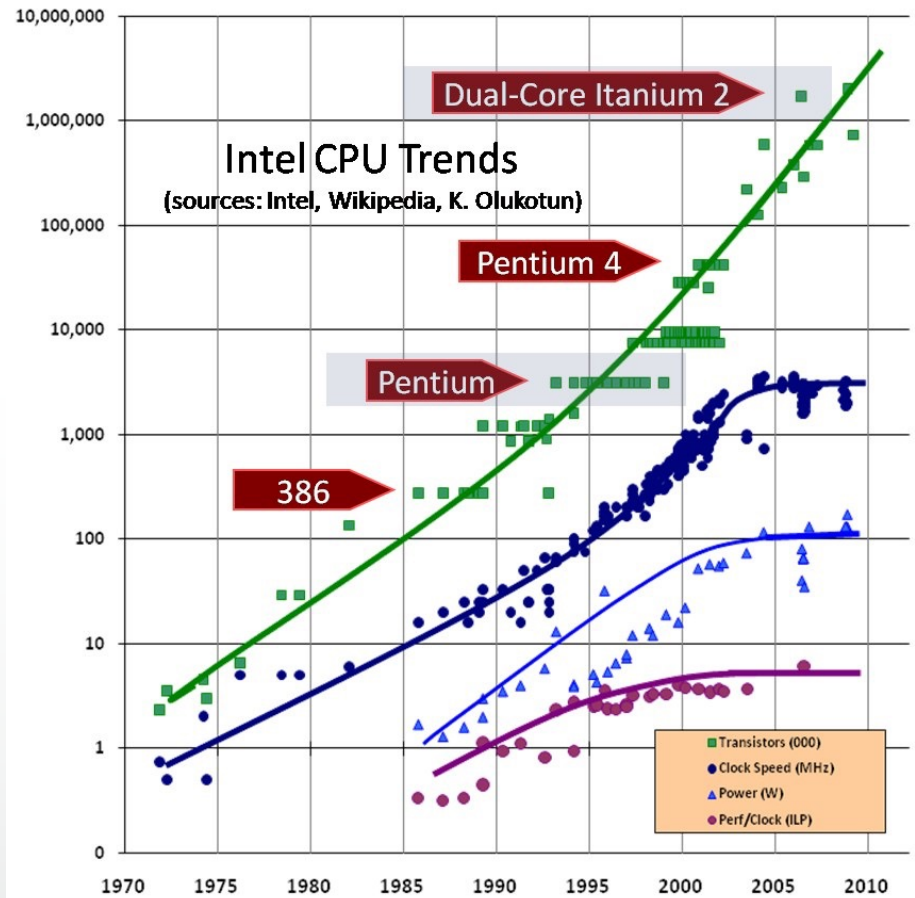


16: Pipelining (1)

Why Pipeline?

- We always want computations to go faster
 - CPU is *literally* the core of any computation
 - Therefore, the obvious solution is to make the CPU go faster
 - But we are reaching (or have reached) the limits of CMOS technology

Moore's Law

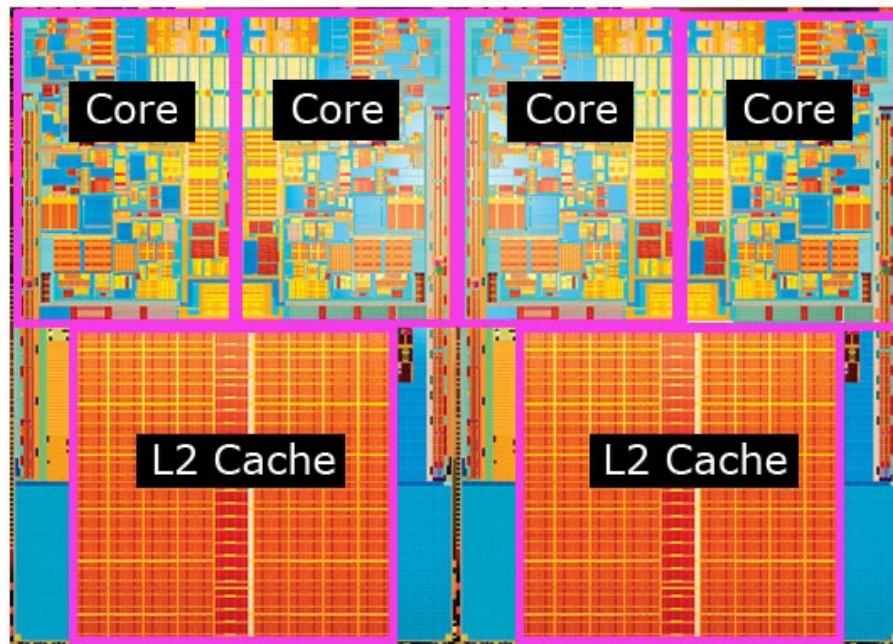


- Clock speed has been constant for the past 2+ decades
- Trend has continued through dual and quad cores

Performance

- If a program doesn't run fast enough on a single core:
 - Use multi-cores
 - Buy more memory
 - Use a novel architecture: GPU, FPGA, etc
- But how do we know what to do?
 - We have to model computation, communication, memory access etc.
 - May want to model both performance and energy

Parallelism



- 4x the computing power
- 4x the performance?
 - Only if we can write the software
 - Parallel programming has been a problem for over 50 years

Performance Metrics

$$Performance = \frac{1}{Execution\ Time}$$

- Execution time (as perceived by the user) is often a combination of user time and system time.

$$CPU\ time = \frac{CPU\ cycles\ for\ program}{Clock\ rate}$$

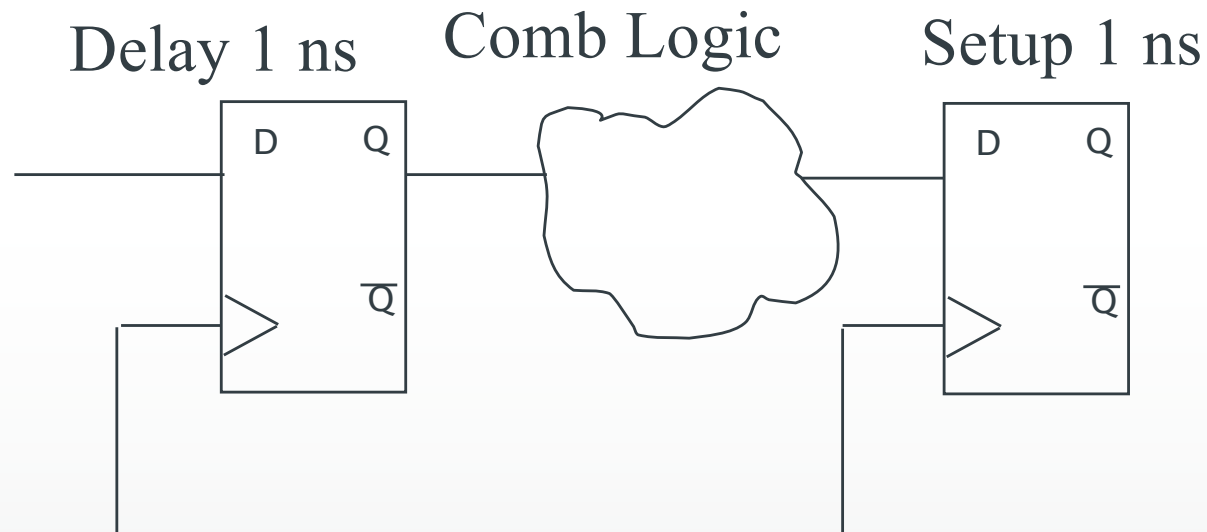
$$CPU\ Clock\ cycles =$$

$$Instructions\ for\ a\ program \times Average\ clock\ cycles\ per\ instruction\ (CPI)$$

Instructions per Cycle

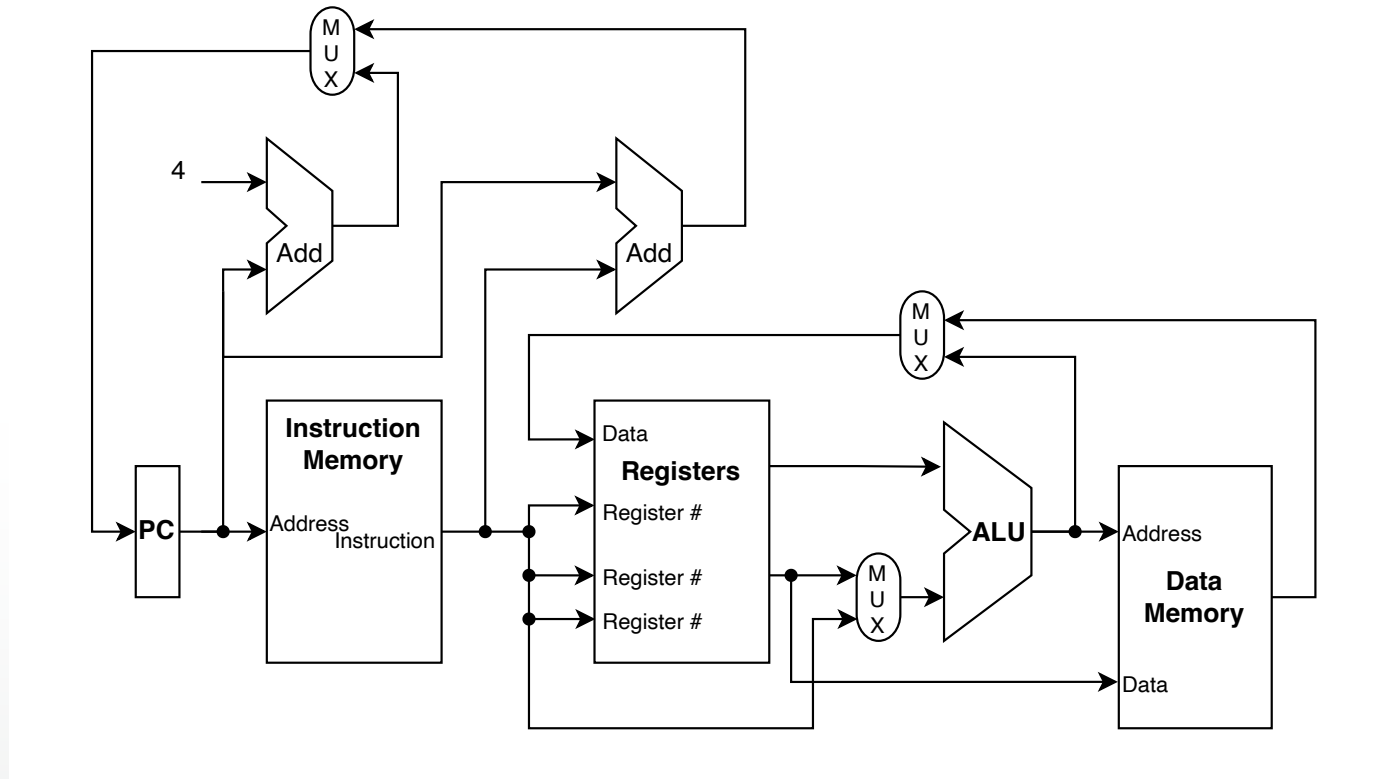
- An important measure of performance is Instructions per Cycle (IPC)
- or the reciprocal, Cycles per Instruction (CPI)
- For a single cycle processor, $IPC = 1$
 - This is good, but the clock speed may be low
 - What limits the clock speed?

Timing Constraints



Clock period is 5 ns,
Clock Frequency is 200 MHz,
Max delay through Comb Logic is 3 ns

RISC-V



- In a single cycle design, the PC increments each clock cycle
- Therefore in one clock cycle, the CPU must fetch and decode an instruction, execute it, access data memory, and write back to the register file.

Example

Instruction Class	Instruction Fetch (IF)	Register Read & Instruction Decode (ID)	ALU Operation (EX)	Data Memory Access (MEM)	Write back to Register (WB)	Total
Load (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format	200 ps	100 ps	200 ps		100 ps	600 ps
Branch	200 ps	100 ps	200 ps			500 ps

Taken from Patterson and Hennessy

Note:

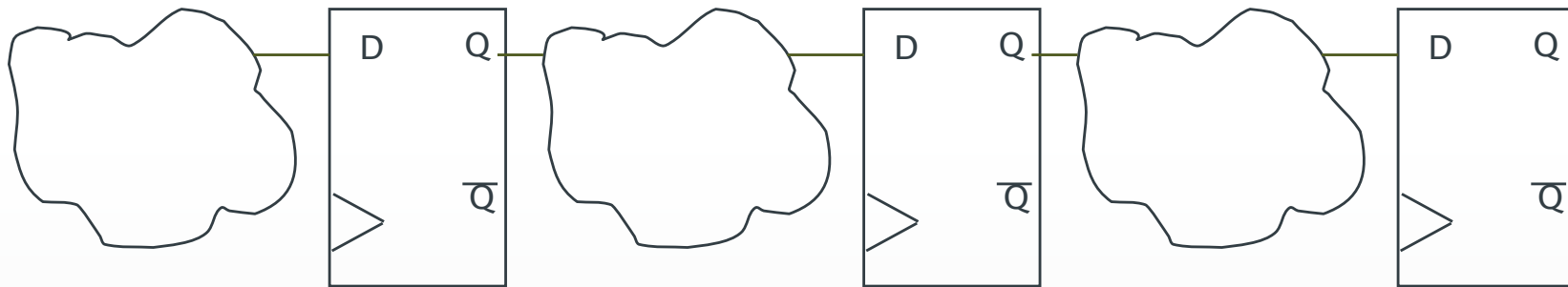
- Different "stages" take different times to complete (this example is not necessarily realistic)
- Only the load instruction uses all stages

Example (continued)

- The *latency* is the maximum time to complete an operation
 - In this case, load takes 800 ps
- In a single cycle CPU, this means that the *minimum* clock *period* must be 800 ps.
- The *maximum* clock *frequency* is therefore
 - $1/800 \text{ ps} = 1.25 \text{ GHz}$

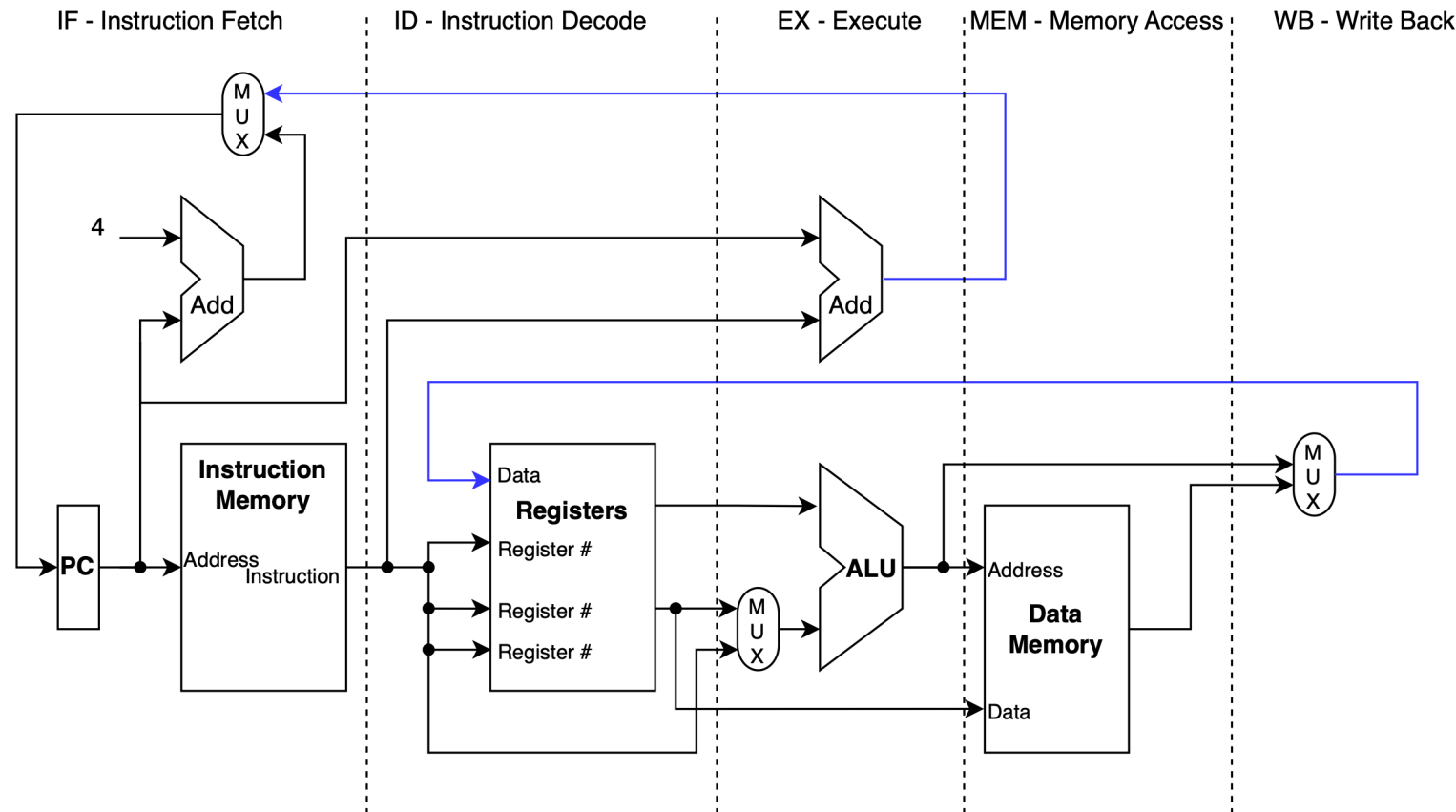
Pipelining – recap

- If delay through combinational logic is too great, logic can be divided:



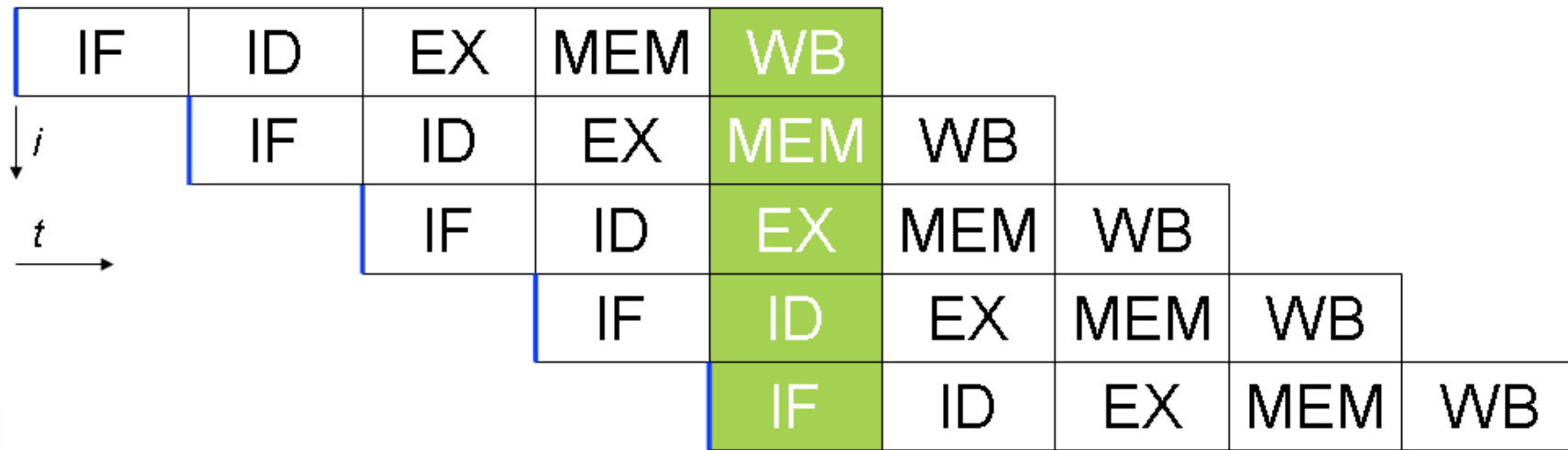
- Clock speed is greater; time for one piece of data to go through (latency) is greater; throughput is also greater – one new result per clock cycle.
- Efficient if the pipeline is kept full

Pipelined RISC-V



- 5 stages (as earlier)
 - Dashed lines are clock boundaries

Classic 5 stage RISC pipeline



http://en.wikipedia.org/wiki/Classic_RISC_pipeline

- Instruction Fetch
- Instruction Decode
- EXecute
- MEMory access
- register Write Back

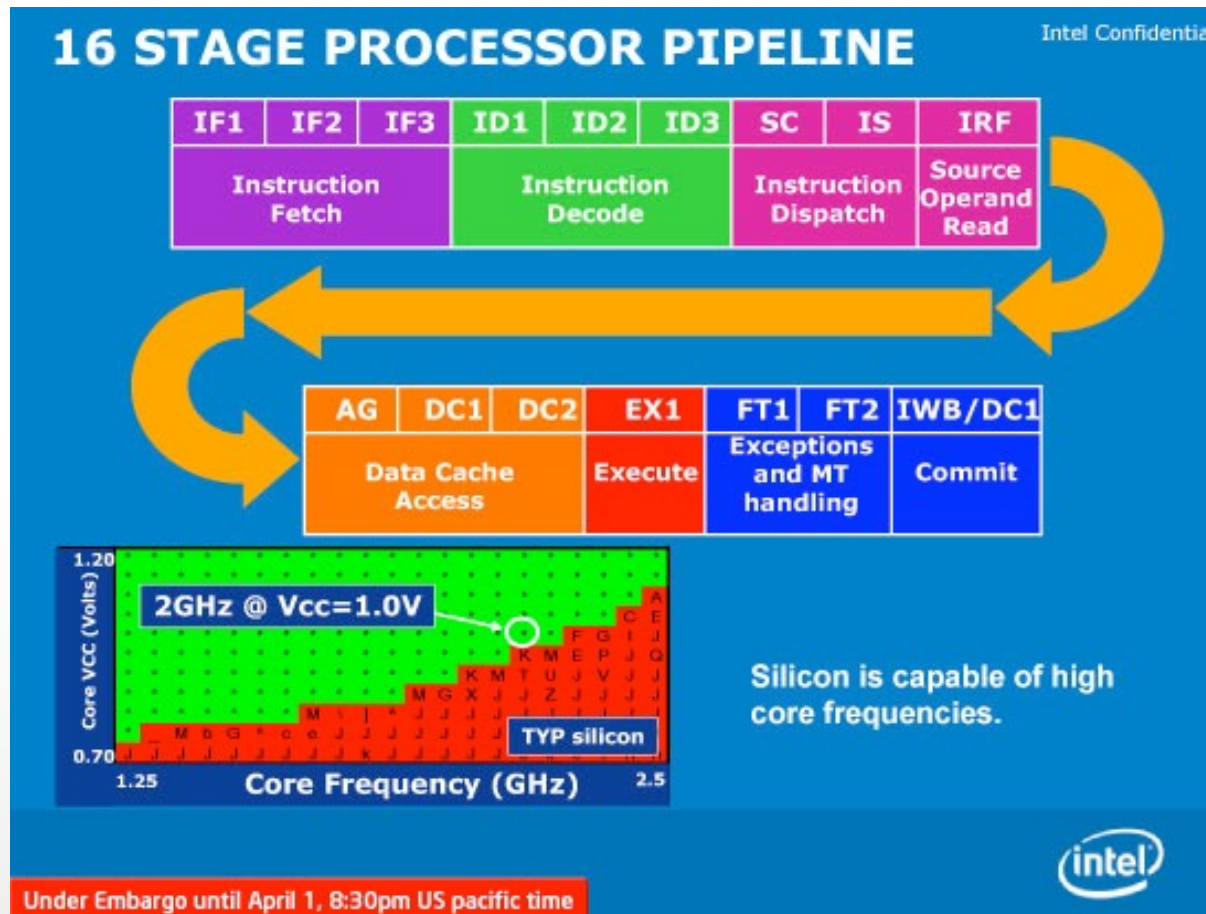
RISC-V was designed with this model in mind
IBEX has 2 stages, other versions have 4 or 6 stages etc.

How many stages can we have?

Year	CPU	MHz	Transistors	Process	Pipeline
1978	8086	4.77	29K	3000nm	2
1982	80286	8	134K	1500nm	3
1985	80386	16	275K	1000nm	3
1989	80486	25	1.2M	1000nm	5
1993	Pentium	60	3.1M	800nm	5
1995	Pentium Pro	150	7.5M	600nm	12
1997	Pentium II	233	7.5M	350nm	12
1999	Pentium III	450	9.5M	250nm	12
2000	Willamette P4	1400	42M	180nm	20
2002	Northwood P4	2000	55M	130nm	20
2004	Prescott P4	3400	125M	90nm	31
2001	Itanium	733	25M	180nm	10
2002	Itanium 2	1000	220M	180nm	8

- Intel processors
- What's actually in 12/20/31 stages?

Superpipelining



- Intel Atom
- 6 colours!
- Each operation is divided
- Each stage executes a micro-op (not an instruction)
- More simple stages faster clock, but greater likelihood of stalls (see later)

Pipelined execution

- Floating point operations can take several clock cycles
 - Obvious candidate for pipelining
- Even integer arithmetic can take 2-3 cycles.
- History of Intel processors suggests that there are limited benefits to going too deep.

Array multiplier

